Практическая работа №25
Работа с локальной БД в приложении

1 Цель работы

1.1 Изучить принципы работы с БД в приложениях на Avalonia UI.

2 Литература

2.1 Avalonia documentation – Текст : электронный // AvaloniaUI, 2024. – URL: https://docs.avaloniaui.net/

3 Подготовка к работе

3.1 Повторить теоретический материал (см.п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

5.1 Создайте новый проект Avalonia

5.1.1 Установите пакеты Microsoft.EntityFrameworkCore и Microsoft.EntityFrameworkCore.Sqlite

5.1.2 Создайте контекст базы данных AppDbContext

```
public class AppDbContext : DbContext
{
    public DbSet<User> Users { get; set; }

    protected override void
OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        string dbPath = GetDatabasePath("app.db");
        optionsBuilder.UseSqlite($"Data
Source={dbPath}");
    }

    private static string GetDatabasePath(string
fileName)
    {
        string folder =
Environment.GetFolderPath(Environment.SpecialFolder.Loc
alApplicationData);
        return Path.Combine(folder, fileName);
    }
}
```

5.1.3 В классе User определите свойства Id, Login, Password

5.1.4 Также при создании экземпляра объекта контекста добавьте строчку

Database.EnsureCreated() для того, чтобы создать файл БД при его отсутствии.

5.2 Взаимодействие с контекстом данных

5.2.1 Для взаимодействия с контекстом данных создайте класс UserService, и в нем создайте методы для работы с пользователями.

5.2.2 Контекст данных и сервис зарегистрировать в приложении либо напрямую передавать во ViewModel

5.3 Пользовательский интерфейс для взаимодействия с БД

5.3.1 На странице приложения добавить поля ввода логина и пароля, кнопку создать и список пользователей.

5.3.2 При вводе данных в поля ввода и нажатие кнопки сохранять пользователя в БД и отображать его в списке.

5.3.3 Добавить для пользователей в списке кнопку «Удалить» при нажатии на которую пользователь удаляется из БД и списка.

5.4 Редактирование

5.4.1 Добавить в приложение возможность сохранения пользователя: при нажатии на кнопку «Редактировать» данные о пользователе загружаются в поля ввода, при нажатии на кнопку «Сохранить» должны обновляться данные в БД в соответствии с изменениями.

6 Порядок выполнения работы

6.1 Выполнить все задания из п.5.

6.2 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 Как осуществляется работа с БД в приложениях на Avalonia?

8.2 Для каких целей в приложениях используются БД SQLite?

Практическая работа №26
Получение данных по сети

1 Цель работы

1.1 Изучить процесс разработки приложений, использующих API, на Avalonia UI

1.2 Закрепить навыки разработки приложений на Avalonia UI.

2 Литература

2.1 Avalonia documentation – Текст : электронный // AvaloniaUI, 2024. – URL: https://docs.avaloniaui.net/

3 Подготовка к работе

3.1 Повторить теоретический материал (см.п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

5.1 Получение доступа к API

5.1.1 Зарегистрируйтесь на сайте OpenWeatherApi

5.1.2 Перейдите в профиль пользователя и скопируйте себе Api-ключ

5.2 Разработка методов доступа к API

5.2.1 Создайте форк и клонируйте репозиторий
https://github.com/ReyRom-Edu/WeatherApp.git

5.2.2 В файле appsettings.json указать ваш ключ API (не отправляйте файл с ключем в репозиторий!)

5.2.3 Изучите файлы проекта, ознакомьтесь с WeatherService.

5.2.4 В классе реализуйте методы:

- метод на основе названия города осуществляет запрос к Current weather Api (https://openweathermap.org/current#name) и получает информацию о текущей погоде
- метод осуществляет запрос к Geocoding Api (https://openweathermap.org/api/geocoding-api) и по названию города получает список подходящих городов и их географические координаты
- метод на основе географических координат города осуществляет запрос к Current Weather Api (https://openweathermap.org/current) и получает информацию о текущей погоде

При запросах используйте метрическую систему и русский язык

5.3 Отображение информации

5.3.1 Добавьте на страницу WeatherView поле ввода названия города и кнопку. При нажатии на кнопку на странице отображается карточка с информацией о текущей погоде в городе.

5.3.2 Карточку настроить внутри заготовленного DataTemplate на странице. Карточка должна содержать:

- название города
- описание погоды
- температуру
- как ощущается температура
- давление
- влажность
- направление и скорость ветра
- информацию о дожде / снеге / облачности

Все данные должны отображаться в понятном для пользователя виде (понятные единицы измерения и подписи данных, при необходимости используйте конверторы), например:

- Давление в миллиметрах ртутного столба
- Направление ветра на основе сторон света (северный, северо-западный и т.д.)
- Метрические единицы измерения для величин

Обязательно указывайте единицы измерения параметров. Используемые по умолчанию единицы измерения величин можно узнать по ссылке: https://openweathermap.org/weather-data

5.4 Добавьте на карточку иконку соответствующую погоде (https://openweathermap.org/weather-conditions#Icon-list)

http://openweathermap.org/img/wn/10d@2x.png

Иконки получайте при помощи метода LoadFromWeb класса ImageHelper

5.4.1 Для отображения карточек используйте привязку к свойству Task:

```
public Task<Bitmap?> Image => ImageHelper.LoadFromWeb(url);
```

Оператор ^ позволяет осуществить привязку к результату асинхронной операции.

```
<Image Source="{Binding Image^}"/>
```

6 Порядок выполнения работы

6.1 Выполнить все задания из п.5.

6.2 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 Как десериализовать JSON-ответ от API

8.2 Как выполнить привязку к результату Task.

8.3 Как работает класс ImageHelper, предоставленный преподавателем?

Практическая работа №27
Разработка приложения

1 Цель работы

1.1 Закрепить навыки разработки приложений на Avalonia UI.

2 Литература

2.1 Avalonia documentation – Текст : электронный // AvaloniaUI, 2024. – URL: https://docs.avaloniaui.net/

3 Подготовка к работе

3.1 Повторить теоретический материал (см.п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

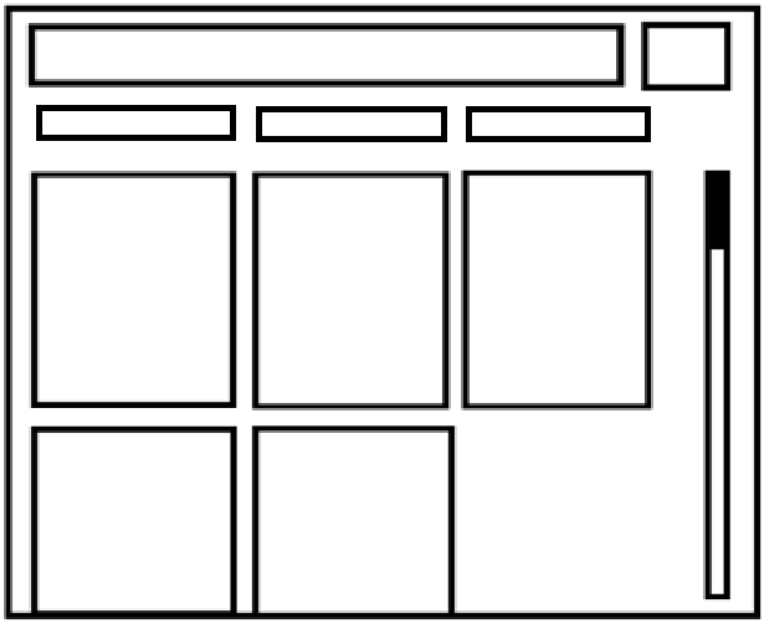
5 Задание

В работе используется приложение из ПР №26

5.1 Модификация интерфейса приложения

5.1.1 На странице расположите поле ввода, кнопку Поиск, список выбора городов и список прогнозов погоды.

5.1.2 Прогноз погоды должен отображаться в виде карточки, карточки отображаются в виде списка. Общий вид интерфейса должен соответствовать макету:



Также учтите адаптивность интерфейса под мобильные устройства

5.1.3 Для списка карточек используйте ItemsControl. В качестве значения для ItemsControl.ItemsPanel установите WrapPanel.

```
<ItemsControl.ItemsPanel>
    <ItemsPanelTemplate>
        <WrapPanel/>
    </ItemsPanelTemplate>
</ItemsControl.ItemsPanel>
```

5.1.4 ItemsControl расположите внутри ScrollViewer

5.2 Добавление записей в список

Пользователь должен вводить название города в поле ввода, нажимать на кнопку «Поиск» и после этого должен заполняться список городов на основе значений, которые вернуло API. Пользователь выбирает один из городов, который будет добавлен в список прогнозов погоды.

5.3 Сохранение данных

После добавления новой карточки необходимо сохранить на устройстве список городов, карточки которых отображаются в приложении

5.4 Для сохранения данных требуется использовать класс Preferences в проекте. Ознакомьтесь с его содержимым.

5.4.1 Для сохранения данных воспользоваться методом

```
await Preferences.Save("ключ", данные)
```

5.4.2 При запуске приложения загружайте данные из списка при помощи метода

```
var data = await Preferences.Load("ключ", знач_по_умолч)
```

5.4.3 Для выполнения асинхронных операций в конструкторе ViewModel используйте

```
TaskNotifier? _initializationTask;
public Task? InitializationTask
{
    get => _initializationTask;
    set     =>     SetPropertyAndNotifyOnCompletion(ref
_initializationTask, value);
}

public WeatherViewModel(WeatherService weatherService)
{
    _weatherService = weatherService;

    InitializationTask = Task.Run(async () =>
    {
        //Асинхронный код
    });
}
```

5.5 Обновление данных

5.5.1 Добавить на карточку кнопки «Обновить» и «Удалить»

5.5.2 При нажатии «Удалить» удаляйте карточку из списка прогнозов

5.5.3 При нажатии «Обновить» обновляйте значения данных прогноза до актуальных на текущий момент.

6 Порядок выполнения работы

6.1 Выполнить все задания из п.5.

6.2 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 Как использовать асинхронные операции при инициализации объекта?

8.2 Как работает класс Preferences, предоставленный преподавателем?